

### 1.2.1. Pass or Fail

24:29

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

### Input Format:

- The first input will be an integer  $n$ , the number of courses.
- The second input will be  $n$  integers representing the marks of the student in each of the  $n$  courses, separated by a space.

### Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

**Note:**

- **Aggregate Percentage:** Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

## Sample Test Cases



 **SUBMIT**

3.40 ms

5.00 ms

5 out of 5 shown test case(s) passed

✔ 5 out of 5 hidden test case(s) passed

✓ Test case 1 5 ms



### Expected output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: -First-Division

✓ Test case 2 3 ms

 Terminal

Test cases

Course

https://mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5f/698c54860aba96214c8b7d61/6682597fa298...

Chat

CODETANTRA

Home Learn Anywhere

202501100230@mitaoe.ac.in Support Logout

1.2.2. Fibonacci Series07:21

Write a Python program that uses recursion to print the first  $n$  terms of the Fibonacci series.

Input Format:

A single integer  $n$  representing the number of terms to generate.

Output Format:

A single line containing the first  $n$  terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

fibonacci...

```
1 def fibonacci(n):
2
3     if n <= 2:
4         return n-1
5
6     else:
7         return fibonacci(n-1) + fibonacci(n-2)
8
9
10 n = int(input())
11 for i in range(1, n+1):
12     print(fibonacci(i), end=" ")
13
```

Average time0.007 s7.25 ms

Maximum time0.018 s18.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 14 ms

Expected output

0 1 1 2 3

Actual output

0 1 1 2 3

Test case 23 ms

Terminal

Test cases

< PREV

RESET

SUBMIT

NEXT >

Course

https://mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5f/698c54860aba96214c8b7d61/6774c521f4ab...

CODETANTRA

HomeLearn Anywhere

202501100230@mitaoe.ac.inSupportLogout

1.2.3. Pattern - 103:26

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format

The output should display the pattern of asterisks (\*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

rightangl...

```
1 #Type Content here...
2
3 n = int(input())
4
5 for i in range(1, n+1):
6     print("*" * i)
```

Average time0.002 s2.17 ms

Maximum time0.003 s3.00 ms

2 out of 2 shown test case(s) passed4 out of 4 hidden test case(s) passed

Test case 13 msDEBUG

Expected output

Actual output

TerminalTest cases

< PREV

RESET

SUBMIT

NEXT >



Course

https://mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5f/698c54860aba96214c8b7d61/6774cc86f4ab...

Chat

CODETANTRA

[Home](#) [Learn Anywhere](#)

202501100230@mitaoe.ac.in Support [Logout](#)

1.2.4. Pattern - 203:29

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format:

The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

Refer to the displayed test cases for the sample pattern.

Sample Test Cases

numberP...

```
1 #Type-Content-here...
2 n = int(input())
3
4 for i in range(1, n+1):
5     for j in range(1, i+1):
6         print(j, end=" ")
7     print("")
8
```

Average time0.002 s2.25 ms

Maximum time0.003 s3.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 13 ms

DEBUG

Expected output

Actual output

5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

1 2 3 4 5

Terminal

Test cases

< PREV

RESET

SUBMIT

NEXT >



### 2.2.1. Linear search Technique

Write a program to check whether the given element is present or not in the array of elements using linear search.

**Input format:**

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

### Output format:

- If the element is found, print the index. If the element found multiple times, print the starting index
- If the element is not found, print **Not found**.

### Sample Test Case:

**Input:**

1 2 3 4 3 5 6

3

**Output:**

2

## Sample Test Cases

+

 Explorer

CTP1709...

 **SUBMIT**


**Debugger**

Average time  
**0.003 s**  
3.00 ms

Maximum time  
**0.004 s**  
4.00 ms

✔ 2 out of 2 shown test case(s) passed

✔ 2 out of 2 hidden test case(s) passed

✓ Test case 1 4 ms **DEBUG**

### Expected output

1 2 3 4 3 5 6

3

Actual output

1 2 3 4 3 5 6

3

✓ Test case 2 2 ms

**> Terminal**

Test cases

[< PREV](#)
[RESET](#)
[SUBMIT](#)
[NEXT >](#)

Course

https://mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d62/698c54860aba96214c8b7d64/6774d0abf4a...

CODETANTRA

HomeLearn Anywhere

202501100230@mitaoe.ac.inSupportLogout

2.2.2. Captain of the Team04:17

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

**Input Format:**  
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

**Output Format**  
The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

captainof...

```
1 heights = list(map(int, input().split()))
2
3 tallest_player = max(heights)
4
5 print(tallest_player)
```

Average time0.002 s2.33 msMaximum time0.003 s3.00 ms

1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 13 msDEBUG

Expected output171 169 185 156 174 191 186 190 187 172 160191

Actual output171 169 185 156 174 191 186 190 187 172 160191

TerminalTest cases

< PREV

RESET

SUBMIT

NEXT >

### 3.2.1. Numpy: Matrix Operations

The given code takes two  $3 \times 3$  matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

#### Task:

You are required to compute and display the results of the following matrix operations:

1. Addition (`matrix_a + matrix_b`)
2. Subtraction (`matrix_a - matrix_b`)
3. Element-wise Multiplication (`matrix_a * matrix_b`)
4. Matrix Multiplication (`matrix_a . matrix_b`)
5. Transpose of Matrix A

#### Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

#### Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases

matrixOp...

SUBMIT

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10
11 # Addition
12 print("Addition (A + B):")
13 a = matrix_a + matrix_b
14 print(a)
15 # Subtraction
16 print("Subtraction (A - B):")
17 b = matrix_a - matrix_b
18 print(b)
19 # Multiplication (element-wise)
20 print("Element-wise Multiplication (A * B):")
```

Average time0.009 s9.00 ms

Maximum time0.012 s12.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 112 msDEBUG

Expected output

Actual output

Enter Matrix A:

Enter Matrix A:

1 2 3

1 2 3

4 5 6

4 5 6

7 8 9

7 8 9

Enter Matrix B:

Enter Matrix B:

1 1 1

1 1 1

Terminal

Test cases



3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays02:38

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

**Input Format:**

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

**Output Format:**

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 h_stack = np.hstack((arr1, arr2))
12
13 # Perform vertical stacking (vstack)
14 v_stack = np.vstack((arr1, arr2))
15
16 print("Horizontal Stack:")
17 print(h_stack)
18
19 print("Vertical Stack:")
20
```

Average time0.009 s8.75 ms

Maximum time0.012 s12.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 112 ms

DEBUG

Expected output

Actual output

Enter Array1:

Enter Array1:

1 2 3

1 2 3

4 5 6

4 5 6

7 8 9

7 8 9

Enter Array2:

Enter Array2:

4 5 6

4 5 6

Terminal

Test cases

3.2.3. Numpy: Custom Sequence Generation01:34

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

**Input Format:**

- The user will input three integer values: start, stop, and step, each on a new line.

**Output Format:**

- The program should print the generated sequence based on the input values.

Sample Test Cases

customS...

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9 arr = np.arange(start, stop, step)
10 # Print the generated sequence
11 print(arr)
```

Average time0.005 s5.50 ms

Maximum time0.007 s7.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 17 ms

DEBUG

Expected output

Actual output

3

3

10

10

2

2

[3 5 7 9]

[3 5 7 9]

Test case 24 ms

Terminal

Test cases

### 3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators 03:14

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

### 1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

## 2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array **A**.

### 3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex:  $A_i \text{ OR } B_i$ ).

**Input Format:**

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

### Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

## Sample Test Cases

different...

 SUBMIT

Average time  
**0.005 s**  
4.67 ms

Maximum time  
**0.007 s**  
7.00 ms

1 out of 1 shown test case(s) passed

---

2 out of 2 hidden test case(s) passed

✓ Test case 1 7 ms **DEBUG**

### Expected output

1 2 3 4

5 6 7 8

Element-wise Sum: -6 -8 -10 -12

Element-wise Difference: -4 -4 -4 -4

Element-wise Product:  $-5 \cdot 12 \cdot 21 \cdot 32$

Mean of A: 2.5

**Actual output**

1 2 3 4

5 6 7 8

Element-wise Sum: 6 8 10 12

Element-wise Difference:  $-4 \ -4 \ -4 \ -4$

Element-wise Product: -5 -12 -21 -32

Mean of A: -2.5

Terminal Test cases

< PREV RESET SUBMIT NEXT >



3.2.5. Numpy: Copying and Viewing Arrays

00:52

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

**Input Format:**

- A single line of space-separated integers.

**Output Format:**

- After modifying the view:

Original array after modifying view: `<original_array>`  
View array: `<view_array>`

- After modifying the copy:

Original array after modifying copy: `<original_array>`  
Copy array: `<copy_array>`

Sample Test Cases

copyAnd...

SUBMIT

```
1 import numpy as np
2
3 inputlist = list(map(int, input().split(" ")))
4
5 #Original array
6 original_array = np.array(inputlist)
7
8 #Create a view
9 view_array = original_array.view()
10
11 #Create a copy
12 copy_array = original_array.copy()
13
14 #Modify the view
15 view_array[0] = 99
16 print("Original array after modifying view:", original_array)
17 print("View array:", view_array)
18
19 #Modify the copy
20 copy_array[1] = 88
```

Average time  
0.004 s  
3.50 ms

Maximum time  
0.005 s  
5.00 ms

2 out of 2 shown test case(s) passed  
2 out of 2 hidden test case(s) passed

Test case 1 5 ms DEBUG

Expected output

10 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]

Actual output

10 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]

Terminal Test cases

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting02:43

The given code in the editor takes a single array, array1, as space-separated integers as input from the user. Additionally, it takes the following inputs:

- search\_value: The value to search for in the array.
- count\_value: The value to count its occurrences in the array.
- broadcast\_value: The value to add for broadcasting across the array.

- You need to complete the code to perform the following operations:
- Searching:** Find the indices where search\_value appears in array1 and print these indices.
  - Counting:** Count how many times count\_value appears in array1 and print the count.
  - Broadcasting:** Add broadcast\_value to each element of array1 using broadcasting, and print the resulting array.
  - Sorting:** Sort array1 in ascending order and print the sorted array.

- Input Format:**
- A single line containing space-separated integers representing array1.
  - An integer search\_value represents the value to search for in the array.
  - An integer count\_value represents the value to count in the array.
  - An integer broadcast\_value represents the value to add to each element of the array.

- Output Format:**
- The indices where search\_value occurs in array1.
  - The count of occurrences of count\_value in array1.
  - The array after adding the broadcast\_value to each element.
  - The sorted array.

Sample Test Cases

arrayOpe...SUBMIT

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 indices = np.where(array1 == search_value)[0]
13 print(indices)
14 # Count occurrences in array1
15 count = np.count_nonzero(array1 == count_value)
16 print(count)
17 # Broadcasting addition
18 broadcasted_array = array1 + broadcast_value
19 print(broadcasted_array)
20 # Sort the first array
```

Average time0.006 s6.25 ms

Maximum time0.010 s10.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 110 msDEBUG

Expected output

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

Actual output

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

Terminal

Test cases



### 3.2.7. Student Data Analysis and Operations

26.48

AA

- Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:
- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
  - **Find total students:** Determine the total number of students in the dataset.
  - **Print all student roll numbers:** Extract and print the roll numbers of all students.
  - **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
  - **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
  - **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
  - **Print all subject marks:** Display the marks of all students for each subject.
  - **Find total marks of students:** Compute the total marks for each student across all subjects.
  - **Find the average marks of each student:** Compute the average marks for each student.
  - **Find average marks of each subject:** Compute the average marks for all students in each subject.
  - **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
  - **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
  - **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
  - **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
  - **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
  - **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
  - **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
  - **Find the count of students who scored  $\geq 90$  in each subject:** Count the number of students who scored 90 or more marks in each subject.
  - **Find the count of subjects in which each student scored  $\geq 90$ :** Determine how many subjects each student scored 90 or more marks in.
  - **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
  - **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.

Operatio...

1 import numpy as np

2

3 a=np.loadtxt("Sample.csv",delimiter=',',skiprows=1)

4

5 #1.Print all student details

6 print("All student Details:\n",a)

7

8 #2.print total students

9

10 print("Total Students:",int(a.shape[0]))

11

12 #3.Print all student Roll numbers

13 print("All Student Roll Nos",a[:,0])

14

15 #4.Print subject 1 marks

16 print("Subject 1 Marks",a[:,1])

17

18 #5.print minimum marks of Subject 2

19 print("Min marks in Subject 2",np.min(a[:,2]))

20

Average time0.010 s10.00 ms

Maximum time0.010 s10.00 ms

1 out of 1 shown test case(s) passed

Test case 110 msDEBUG

Expected output

Actual output

TerminalTest cases



#### 4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

### Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

**Note:**

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

## Sample Test Cases

```

1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 df['Total Sales'] = df['Quantity'] * df['Price']
10 df['Date'] = pd.to_datetime(df['Date'])
11 df['Month'] = df['Date'].dt.to_period('M')
12 monthly_sales = df.groupby('Month')['Total Sales'].sum()
13
14 # Find the month with the highest total sales
15 best_month = monthly_sales.idxmax()
16 highest_sales = monthly_sales.max()
17
18 print(f"Best month: {best_month}")
19 print(f"Total sales: ${highest_sales:.2f}")
20

```

Average time  
**0.014 s**  
14.33 ms

Maximum time  
**0.030 s**  
30.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 30 ms

### Expected output

sales\_data.csv

Best month: 2025-01

Total sales: \$1210.00

Actual output

sales\_data.csv

Best-month: 2025-01

Total sales: \$1210.00

 Terminal

Test cases

#### 4.2.2. Best Selling Product

01:35

AA

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

##### Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

##### Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases



#### monthFor...

SUBMIT

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 product_quantity = df.groupby('Product')['Quantity'].sum()
10 # Find the product with the highest total quantity sold
11 best_product = product_quantity.idxmax()
12 highest_quantity = product_quantity.max()
13
14 # Display the result
15 print(f"Best selling product: {best_product}")
16 print(f"Total quantity sold: {highest_quantity}")
17
```

Average time

0.011 s

11.00 ms

Maximum time

0.022 s

22.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 22 ms

DEBUG

Expected output

sales\_data.csv

Best selling product: Product A

Total quantity sold: 23

Actual output

sales\_data.csv

Best selling product: Product A

Total quantity sold: 23

Terminal

Test cases

< PREV

RESET

SUBMIT

NEXT >

#### 4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

### Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

**Note:**

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

## Sample Test Cases

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 city_sales = df.groupby('City')['Quantity'].sum()
10 best_city = city_sales.idxmax()
11
12 # Display the result
13 print(f"City sold the most products: {best_city}")
14
```

Average time  
**0.013 s**  
13.00 ms

Maximum time  
**0.028 s**  
28.00 ms

✔ 1 out of 1 shown test case(s) passed

---

✔ 2 out of 2 hidden test case(s) passed

✓ Test case 1 28 ms

### Expected output

sales\_data.csv

City sold the most products: Los Angeles

Actual output

sales\_data.csv

City sold the most products: Los Angeles

**>\_ Terminal**

Test cases



4.2.4. Most Frequently Sold Product Pairs

06:20 ⌕ ✨ 📄 🔗 -

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

**Explanation:**

**Transactions:**

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B:** Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C:** Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C:** Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

**Note:**

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

frequentl...

🔍 SUBMIT

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 daily_transactions = df.groupby('Date')['Product'].apply(list)
12 pair_counts = Counter()
13
14 for products in daily_transactions:
15     products = sorted(products)
16     pairs = list(combinations(products, 2))
17     pair_counts.update(pairs)
18
19 if pair_counts:
20     max_count = max(pair_counts.values())
```

Average time  
0.011 s  
10.67 ms

Maximum time  
0.022 s  
22.00 ms

🟢 1 out of 1 shown test case(s) passed  
🟢 2 out of 2 hidden test case(s) passed

🟢 Test case 1 22 ms

DEBUG

📄

Expected output

Actual output

sales\_data.csv

sales\_data.csv

Product A and Product B : 2 times

Product A and Product B : 2 times

Product A and Product C : 2 times

Product A and Product C : 2 times

Terminal

Test cases

#### 4.2.5. Titanic Dataset Analysis and Data Cleaning

04:56

⌵ ⚙️ 📄 🔗 −

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

#### Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

#### titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 #Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 #1. Display the first 5 rows of the dataset
8 print(data.head())
9
10 #2. Display the last 5 rows of the dataset
11 print(data.tail())
12
13 #3. Get the shape of the dataset
14 print(data.shape)
15
16 #4. Get a summary of the dataset (info)
17 print(data.info())
18
19 #5. Get basic statistics of the dataset
20 print(data.describe())
```

Average time

0.091 s

91.00 ms

Maximum time

0.091 s

91.00 ms

1 out of 1 shown test case(s) passed

Test case 1

91 ms

DEBUG

#### Expected output

```
... PassengerId ... Survived ... Pclass ... .. Fare ... Cabin ...
Embarked ...
0 ..... 1 ..... 0 ..... 3 ..... 7.2500 ... NaN ...
..... S ...
1 ..... 2 ..... 1 ..... 1 ..... 71.2833 ... C85 ...
..... C ...
2 ..... 3 ..... 1 ..... 3 ..... 7.9250 ... NaN ...
..... S ...
```

#### Actual output

```
... PassengerId ... Survived ... Pclass ... .. Fare ... Cabin ...
Embarked ...
0 ..... 1 ..... 0 ..... 3 ..... 7.2500 ... NaN ...
..... S ...
1 ..... 2 ..... 1 ..... 1 ..... 71.2833 ... C85 ...
..... C ...
2 ..... 3 ..... 1 ..... 3 ..... 7.9250 ... NaN ...
..... S ...
```

Terminal

Test cases



4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

09:04

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
- Convert the 'Sex' column to numeric values (male: 0, female: 1).
- One-hot encode the 'Embarked' column, dropping the first category.
- Get the mean age of passengers.
- Get the median fare of passengers.
- Get the number of passengers by class.
- Get the number of passengers by gender.
- Get the number of passengers by survival status.
- Calculate the survival rate of passengers.
- Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Paisson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

titanicDat...

1 import pandas as pd  
2 import numpy as np  
3  
4 # Load the Titanic dataset  
5 data = pd.read\_csv('Titanic-Dataset.csv')  
6 data['FamilySize'] = data['SibSp'] + data['Parch']  
7  
8 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)  
9 data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)  
10  
11 # 2. Convert 'Sex' to numeric (male: 0, female: 1)  
12 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})  
13  
14 # 3. One-hot encode the 'Embarked' column  
15 data = pd.get\_dummies(data, columns=['Embarked'])  
16  
17 # 4. Get the mean age of passengers  
18 mean\_age = data['Age'].mean()  
19 print(mean\_age)  
20

Average time  
0.038 s  
38.00 ms

Maximum time  
0.038 s  
38.00 ms

1 out of 1 shown test case(s) passed

Test case 1 38 ms

DEBUG

Expected output  
29.69911764705882  
14.4542  
3...491  
1...216  
2...184  
Name: Pclass, dtype: int64

Actual output  
29.69911764705882  
14.4542  
3...491  
1...216  
2...184  
Name: Pclass, dtype: int64

Terminal

Test cases



#### 4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked\_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below.

[illegible]

**Sample Data:**

er  titanicDat...

 SUBMIT

```

1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10 # 1. Calculate the survival rate by class
11 print(data.groupby('Pclass')['Survived'].mean())
12
13 # 2. Calculate the survival rate by embarked location
14 print(data.groupby('Embarked_S')['Survived'].mean())
15
16 # 3. Calculate the survival rate by family size
17 print(data.groupby('FamilySize')['Survived'].mean())
18
19 # 4. Calculate the survival rate by being alone
20 print(data.groupby('IsAlone')['Survived'].mean())

```

Average time  
**0.045 s**  
45.00 ms

Maximum time  
**0.045 s**  
45.00 ms

1 out of 1 shown test case(s) passed

✓ Test case 1 45 ms **DEBUG**

### Expected output

Actual output

Pclass

Pclass

1. ... 0.629630

1 - - - 0.629630

2. . . . 0.472826

2. . . . 0.472826

3. ... 0.242363

3. . . . 0.242363

```
Name: Survived, dtype: float64
```

```
Name: Survived, dtype: float64
```

Embarked Se

Embarked S

 Terminal

Test cases

#### 4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked\_S).
4. Get the number of non-survivors by embarkation location (Embarked\_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

[illegible]

**Sample Data:**

 titanicDat...

```

1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9 # 1. Get the number of survivors by gender
10 print(data[data['Survived']==1]['Sex'].value_counts())
11
12 # 2. Get the number of non-survivors by gender
13 print(data[data['Survived']==0]['Sex'].value_counts())
14
15 # 3. Get the number of survivors by embarked location
16 print(data[data['Survived']==1]['Embarked_S'].value_counts())
17
18 # 4. Get the number of non-survivors by embarked location
19 print(data[data['Survived']==0]['Embarked_S'].value_counts())
20

```

Average time  
**0.038 s**  
38.00 ms

Maximum time  
**0.038 s**  
38.00 ms

✔ 1 out of 1 shown test case(s) passed

✓ Test case 1 38 ms **DEBUG**

### Expected output

```
female...233
male...109
Name: Sex, dtype: int64
male...468
female...81
Name: Sex, dtype: int64
1...217
```

Actual output

```
female....233
male.....109
Name: Sex, dtype: int64
male.....468
female....81
Name: Sex, dtype: int64
1.....217
```

**> Terminal**

Test cases



### 5.2.1. Titanic Dataset

18.32

AA

Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:

#### Dataset Information:

The dataset is stored in a CSV file named `titanic.csv` and has been loaded using the `pandas` library. It contains the following columns:

- `Pclass`: Passenger class (1 = First, 2 = Second, 3 = Third).
- `Gender`: Gender of the passenger (male/female).
- `Age`: Age of the passenger.
- `Survived`: Survival status (0 = Did not survive, 1 = Survived).
- `Fare`: Ticket fare paid by the passenger.

#### Visualization:

To represent these trends, you will create 5 visualizations using Matplotlib. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).

#### Visualization Details:

Write the code to create a series of visualizations as follows:

##### Bar Plot (Pclass Distribution):

- Create a bar plot to show the distribution of passengers across the different passenger classes (`Pclass`).
- Use the color `skyblue` for the bars.
- Title the plot as `"Passenger Class Distribution"`.
- Label the x-axis as `"Pclass"` and the y-axis as `"Count"`.

##### Pie Chart (Gender Distribution):

- Create a pie chart to display the distribution of male and female passengers.
- Use `lightblue` for males and `lightcoral` for females.
- Include percentages on the slices (use `autopct='%1.1f%%'`).
- Title the plot as `"Gender Distribution"`.

##### Histogram (Age Distribution):

- Create a histogram to visualize the distribution of passengers' ages.
- Use `lightgreen` for the bars with black edges (`edgecolor = 'black'`).
- Set the number of bins to 8 for the histogram.

#### titanicDat...

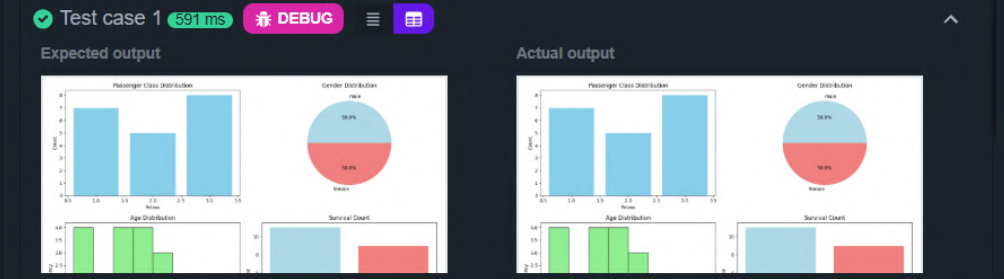
SUBMIT

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset from the CSV file
5 df = pd.read_csv('titanic.csv')
6
7 # Set up the figure for 5 subplots
8 fig, axes = plt.subplots(3, 2, figsize=(12, 12))
9
10 pclass_counts = df['Pclass'].value_counts().sort_index()
11 axes[0, 0].bar(pclass_counts.index, pclass_counts.values, color='skyblue')
12 axes[0, 0].set_title("Passenger Class Distribution")
13 axes[0, 0].set_xlabel("Pclass")
14 axes[0, 0].set_ylabel("Count")
15
16 gender_counts = df['Gender'].value_counts()
17 axes[0, 1].pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%',
18               colors=['lightblue', 'lightcoral'])
19 axes[0, 1].set_title("Gender Distribution")
```

Average time0.591 s591.00 ms

Maximum time0.591 s591.00 ms

1 out of 1 shown test case(s) passed



Terminal

Test cases

### 5.2.2. Histogram of passenger information of Titanic

Write a Python code to plot a histogram for the distribution of the 'Age' column from the Titanic dataset. The histogram should display the frequency of different age ranges with the following specifications:

1. Use **30 bins** for the histogram.
2. Set the **edge color** of the bars to **black (k)**.
3. Label the x-axis as **'Age'** and the y-axis as **'Frequency'**.
4. Add the title **"Age Distribution"** to the histogram.

The Titanic dataset contains columns as shown below.

[illegible]

### Sample Data:

**Note:** Refer to the visible test case for better reference.

## Sample Test Cases

 **Histogra...**

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 plt.hist(data['Age'].dropna(), bins=30, edgecolor='k')
17
18 plt.xlabel("Age")
19 plt.ylabel("Frequency")
20 plt.title("Age Distribution")
```

Average time  
**0.171 s**  
171.00 ms

Maximum time  
**0.171 s**  
171.00 ms

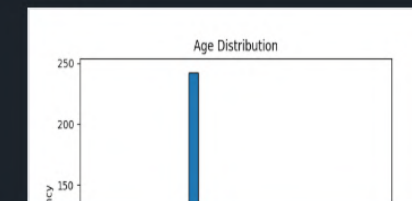
1 out of 1 shown test case(s) passed

✓ Test case 1 171 ms **DEBUG**

### Expected output



Actual output



 Terminal

Test cases



02:20

1. Use the **'Survived'** column to show the count of survivors (0 = Did not survive, 1 = Survived).
2. Set the chart type to **'bar'**.
3. Add the title **"Survival Count"** to the chart.
4. Label the x-axis as **'Survived'** and the y-axis as **'Count'**.

[illegible]

## Sample Test Cases

Average time **0.196 s** Maximum time **0.196 s** **1 out of 1 shown test case(s) passed**  
196.00 ms 196.00 ms

[< PREV](#)
[RESET](#)
[SUBMIT](#)
[NEXT >](#)

#### 5.2.4. Bar Plot for Survival by Gender

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by gender, in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the **'Sex'** column, then use the **value\_counts()** function to count the occurrences of survivors (0 = Did not survive, 1 = Survived) for each gender.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title **"Survival by Gender"** to the chart.
4. Label the x-axis as **'Gender'** and the y-axis as **'Count'**.
5. The legend should indicate **'Not Survived'** and **'Survived'**.

The Titanic dataset contains columns as shown below.

[illegible]

**Sample Data:**

**Note:** Refer to the visible test case for better reference.

## Sample Test Cases

 BarPlotOf...

```
1 '''import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 survival_gender = data.groupby(['Sex', 'Survived']).size().unstack()
17
18
19
20 plt.title("Survival by Gender")
```

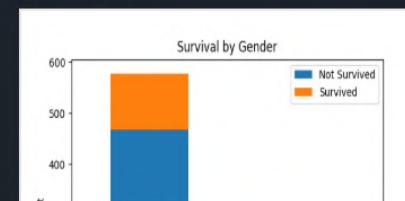
227.00 ms

227.00 ms

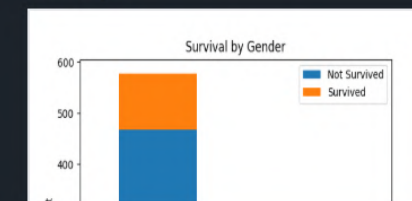
1 out of 1 shown test case(s) passed

✓ Test case 1 227

Expected output



Actual output



 Terminal

## Test cases



### 5.2.5. Bar Plot for Survival by Pclass

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by passenger class (**Pclass**), in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the **Pclass** column and count the number of survivors (0 = Did not survive, 1 = Survived) for each class using **value\_counts()**.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title **"Survival by Pclass"** to the chart.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Count'**.
5. The legend should indicate **'Not Survived'** and **'Survived'**.

The Titanic dataset contains columns as shown below,

[illegible]

**Sample Data:**

**Note:**

- Refer to the visible test case for better reference.
- Ensure you use the `groupby()` function with `value_counts()` to count the survivors and non-survivors for

 BarPlotOf...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Bar Plot for Survival by Pclass
17 survival_pclass = data.groupby(['Pclass', 'Survived']).size().unstack()
18
19 survival_pclass.plot(kind='bar', stacked=True)
```

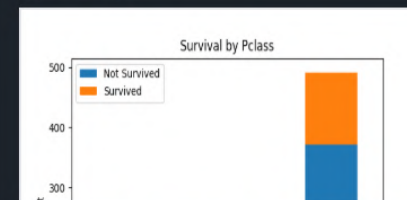
Average time  
**0.219 s**  
219.00 ms

Maximum time  
**0.219 s**  
219.00 ms

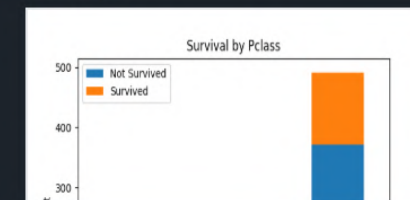
✔ 1 out of 1 shown test case(s) passed

✓ Test case 1 219

### Expected output



**Actual output**



**> Terminal**

Test cases

CODETANTRA  Home  Learn Anywhere ▾

202501100230@mitaoe.ac.in ▾ Support Logout ↗

### 5.2.6. Bar Plot for Survival by Embarked

Write a Python code to plot a stacked bar chart showing the survival count for passengers based on their embarkation location in the Titanic dataset.

The chart should display the following specifications:

1. Use the **Embarked** column to determine the embarkation location. After converting this column into dummy variables (using **pd.get\_dummies()**), plot the survival count based on the **Embarked\_Q** column (representing passengers who embarked from Queenstown) in relation to survival.
2. Set the chart type to 'bar' and make it stacked.
3. Add the title "**Survival by Embarked** " to the chart.
4. Label the x-axis as '**Embarked**' and the y-axis as '**Count**'.
5. Include a legend to distinguish between survivors and non-survivors (label the legend as '**Survived**' and '**Not Survived**').

The Titanic dataset contains columns as shown below,

[illegible]

### Sample Data:

**Note:** Refer to the visible test case for better reference.

 BarPlotOf...

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Bar Plot for Survival by Embarked
17 survival_embarked = data.groupby(['Embarked_Q', 'Survived']).size().unstack()
18 survival_embarked.plot(kind='bar', stacked=True)
19
20 plt.title("Survival by Embarked")
21 plt.xlabel("Embarked")
22 plt.ylabel("Count")
23
24 plt.legend(["Not Survived", "Survived"])
25
26 plt.show()
27

```



### 5.2.7. Box plot for Age Distribution

01:36

⌕

🔍

📄

🔗

—

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Age by Pclass"**.
3. Remove the default subtitle with **plt.suptitle("")**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

#### Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

**Note:** Refer to the visible test case for better reference.

Sample Test Cases



#### BoxPlotF...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Box Plot for Age by Pclass
17 data.boxplot(column='Age', by='Pclass')
18
19 plt.title("Age by Pclass")
20 plt.suptitle('')
```

Average time  
**0.219 s**  
219.00 ms

Maximum time  
**0.219 s**  
219.00 ms

1 out of 1 shown test case(s) passed

Test case 1

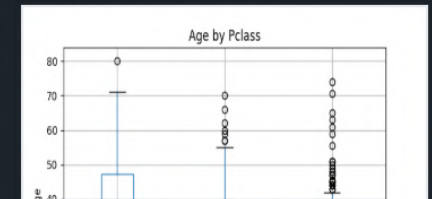
219 ms

DEBUG

Expected output



Actual output



Terminal

Test cases

5.2.8. Box Plot for Age by Survived

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset based on whether passengers survived or not. The boxplot should display the following specifications:

- Use the **Survived** column to group the data for the boxplot (0 = Did not survive, 1 = Survived).
- Set the title of the plot to **"Age by Survival"**.
- Remove the default subtitle with **plt.suptitle("")**.
- Label the x-axis as **'Survived'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

+

BoxPlotF...

01:38

SUBMIT

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Box Plot for Age by Survived
17 data.boxplot(column='Age', by='Survived')
18
19 plt.title("Age by Survival")
20 plt.suptitle('')
```

Average time: 0.216 s, 216.00 ms

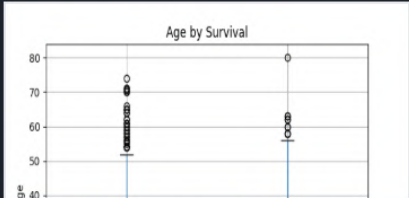
Maximum time: 0.216 s, 216.00 ms

1 out of 1 shown test case(s) passed

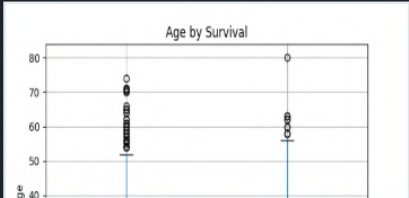
Test case 1 216 ms

DEBUG

Expected output



Actual output



Terminal

Test cases



### 5.2.9. Box Plot for Fare by Pclass

Write a Python code to plot a boxplot that shows the distribution of the 'Fare' column from the Titanic dataset based on the passenger class (Pclass). The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Fare by Pclass"**.
3. Remove the default subtitle with **plt.suptitle("")**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below.

[illegible]

**Sample Data:**

**Note:** Refer to the visible test case for better reference.

## Sample Test Cases

BoxPlotF...

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Box Plot for Fare by Pclass
17 data.boxplot(column='Fare', by='Pclass')
18
19 plt.title("Fare by Pclass")
20 plt.suptitle('')

```

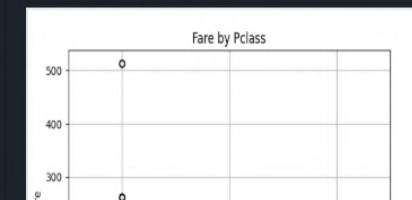
Average time  
**0.215 s**  
215.00 ms

Maximum time  
**0.215 s**  
215.00 ms

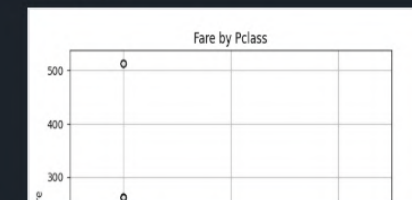
1 out of 1 shown test case(s) passed

✓ Test case 1 215 ms **DEBUG**

### Expected output



Actual output



**> Terminal**

#### Test cases

#### 5.2.10. Scatter Plot for Age vs. Fare

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Set the title of the plot to "**Age vs. Fare**".
3. Label the x-axis as '**Age**' and the y-axis as '**Fare**'.

The Titanic dataset contains columns as shown below.

[illegible]

### Sample Data:

**Note:** Refer to the visible test case for better reference.

## Sample Test Cases

 AgeFareS...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Box Plot for Fare by Pclass
17 plt.scatter(data['Age'], data['Fare'])
18
19 plt.title("Age vs. Fare")
20 plt.xlabel("Age")
```

Average time  
**0.156 s**  
156.00 ms

Maximum time  
**0.156 s**  
156.00 ms

1 out of 1 shown test case(s) passed

✓ Test case 1 156

### Expected output



**Actual output**



 Terminal

Test cases



5.2.11. Scatter Plot for Age vs. Fare by Survived

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset, with points color-coded by survival status. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Color the points based on the **Survived** column: **Red** for passengers who did not survive (**Survived = 0**). **Blue** for passengers who survived (**Survived = 1**).
3. Set the title of the plot to **"Age vs. Fare by Survival"**.
4. Label the x-axis as **'Age'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

AgeFareS...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Scatter Plot for Age vs. Fare by Survived
17 colors = data['Survived'].map({0: 'red', 1: 'blue'})
18
19 plt.scatter(data['Age'], data['Fare'], c=colors)
20
```

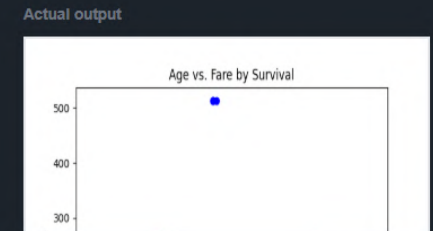
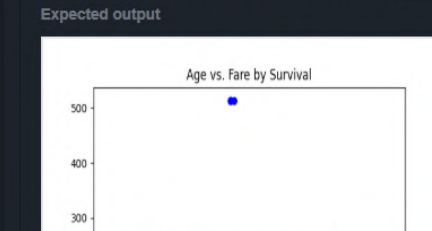
Average time0.169 s169.00 ms

Maximum time0.169 s169.00 ms

1 out of 1 shown test case(s) passed

Test case 1169 ms

DEBUG



Terminal Test cases